Individual e-Portfolio Submission

Module: Intelligent Agents | Candidate: Abdulrahman Husain Ahmed Alhashmi

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

Contents

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

1) Overview

This e-portfolio documents my work across the Intelligent Agents module, my role and learning within the Unit 6 team project, and the skills I developed in designing, evaluating, and reasoning about agent-based systems. It presents a 1,000-word reflective narrative using Rolfe's "What? So what? Now what?" model, evidence of technical outputs (code, designs, forum inputs), a brief evaluation of the Unit 6 artefact, and a mapping to learning outcomes. All screenshots show individual contributions and are excluded from the word count.

2) Reflective

2.1 WHAT — brief account of the project outcomes

I tackled the Unit 6 project with one main aim: build a small agent that could gather, handle, and send out structured data, all while creating a clear record of its steps. The team settled on a simple pipeline with separate parts gathering, handling, checking, and sending—so each part could be checked and tested alone. We pulled in static pages using a simple tool. For dynamic pages, we took a different route to stop browser tools from messing with the main code. A slim loop set clear tasks (fetch sources, parse records, check output, send results) and noted when each one went from waiting to done. I handled keeping these lines sharp and made proof items like run logs, check reports, and sent files.

My practical outputs included a small acquisition script that parses a local HTML table, so the remains reproducible; a processing module that reshapes the data into a consistent schema; validation checks that summarise row counts and missing values; and exporters that write CSV/XLSX for marking. I added quick notes to screen shots, with times and settings, since later readers need to know what worked and under what setup. I also joined team talks on limits. We chose to hide Selenium behind a tight link and show it only when needed, so driver issues would

not mess up the main flow. Each choice seemed minor, but as a group they made the project solid for quick tests.

Past the code, I stressed clear talk. In forum notes, I posted tiny before and after logs of the agent loop, so others could link belief changes and task finishes to real results. I kept notes brief and tested them on set inputs for solid replies. This built up to sharp feedback more on facts than likes so fixes came faster.

2.2 SO WHAT — analysis of experience, decisions, and team dynamics

Our mixed scraping approach struck a good balance between wide coverage and steady results. Static pages delivered quick, reliable data. Selenium let us reach more spots, though it slowed things down and added risks. We tucked Selenium away behind a simple interface. This cut down failure effects and simplified swaps. I picked up a key lesson: put shaky input/output tasks on the edges and keep the main agent cycle solid.

SPADE made us view things through beliefs and aims. This way of thinking changed how we fixed bugs. Rather than just noting a failed step, we checked which belief stayed off and which goal hung open. It sharpened our thoughts and made logs clearer. In a small setup, extra layers can pile up fast. We slimmed down the behaviour part, zeroing in on handling goals and simple message swaps.

For teamwork, regular chats and sharing rough drafts built real speed. When jobs overlapped without clear links noted, we ended up redoing stuff. We fixed that by adding short decision notes beside each chart or code bit. This easy trick smoothed pass offs and cleared up mix ups.

On the emotional side, we swung from sure footed design trust to wary steps when lively pages wrecked our scrapers. That push pull drove us to log every outside call and stick to sure fire sources for show and tells. I got better at showing a rock-solid route, while keeping test branches out in the open for later use.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

2.3 NOW WHAT — changes in practice and forward plan

Next time I build a small agent pipeline, I'll make three quick changes. First, I'll add a short contract test for the data pickup layer. Using sample HTML, it must output a set record list and time within limits. This spots errors fast without big tools. Second, I'll bundle checks and output into one easy call. It returns clear data like counts, ranges, paths, and hashes. That cuts extra code and keeps results the same across jobs. Third, I'll treat log format as a key part. Each line gets time, step, aim, and short details. Steady logs speed up team checks and my own reviews. Patterns stand out better.

I also aim to learn more about agents thinking past a basic goal cycle. I'll try turning beliefs into clear data forms. And test simple plans with unknowns—in a tight so proof stay clear. Limits help they push me to explain each part's value. At the same time, I'll build better team habits. Share rough drafts soon. Ask one clear question per share. Note tiny choices close to the work. These take little time but pay off big.

For real use, I can apply this in two ways. In heavy data classes, I'll plug the same pickup process check output base into a fresh area. Just change the reader. In research jobs, I'll use the log style and proof pack to show trust to non-tech folks. A single page report with counts, basic checks, and file hashes gives a straight tale. The main point? Smart systems need more than self-run. They need clear tracks. Keep actions slim, links tight, and proof plain. That way, agents build trust and show results well.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

2.4 Professional skills matrix

Scale: 1 = novice, 2 = basic, 3 = competent, 4 = strong, 5 = advanced

Competency	Current (1–5)	Evidence to date	Key actions	Success metrics
Agent based computing (BDI/SPADE)	3	Thin goal loop; logged goal transitions; UML roles defined	Model beliefs as typed data; add plan selection for two scenarios; read one overview paper	Two scenarios with plan selection; brief note linking design to outputs
Python engineering (pandas, packaging, logging)	3	Parsing + validation; exports; baseline logging	Refactor into package; uniform log format; docstrings; type hints	Lint + type check pass; uniform logs across three runs
Web acquisition (static/dynamic)	3	Static parser stable; Selenium isolated; retries added	Add explicit waits; headless profile; static fallback for	Three consecutive dynamic runs with full record count
Data quality & reproducibility	3	Row counts, null checks, export hashes	Wrap validation+export as one callable; seed inputs; capture env info	Three reruns with identical counts and hashes
Software design (modularity/UML)	3	AgentController, Scraper, Processor, Storage roles	Clarify interfaces; class docstrings; activity diagram for error paths	Updated UML; peer review note attached
Collaboration & communication	3	Forum posts; decision notes near artefacts	Weekly learning log: one page run report per sprint	Two logs posted; feedback quoted
Ethics and academic integrity	4	Own artefacts; captions with context/time; citations	Privacy checklist for sources; citation cross check	Zero integrity flags; references validated
Project management (scope, risk, timeboxing)	3	Short sprints; scope guard for Selenium	Risk register; entry/exit criteria	Risks tracked; criteria met on time

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

3) Evidence of work

3.1 Answers to exercises, coding scripts, and system designs

Figure 1: BDI model overview

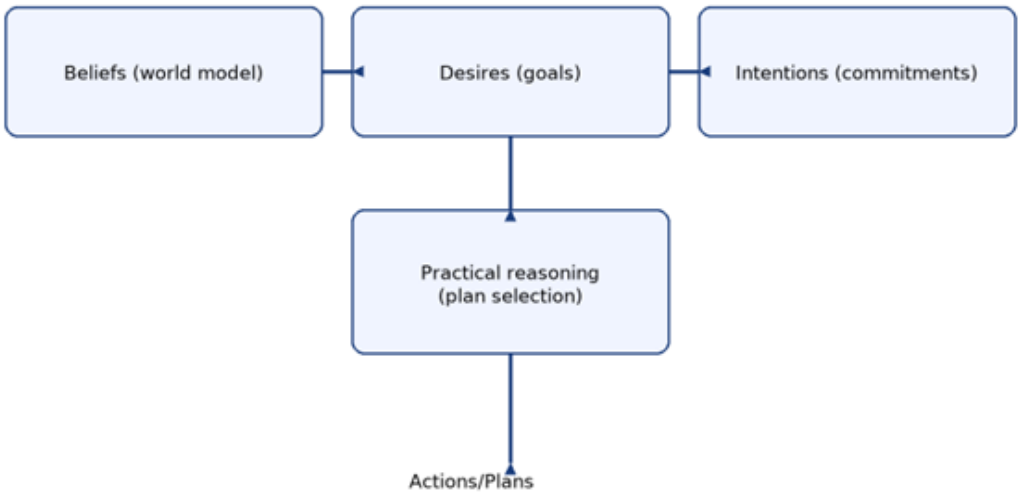

Figure 1: BDI model overview

Figure 2: Agent pipeline for data acquisition and processing

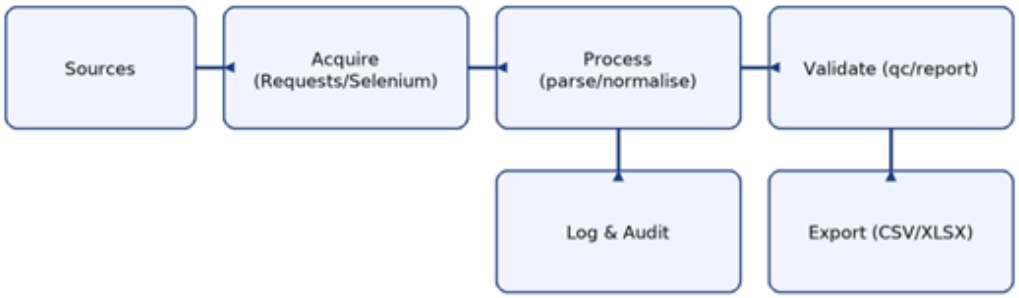

Figure 2: Agent pipeline for data acquisition and processing

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

Figure 3: UML sketch of core components

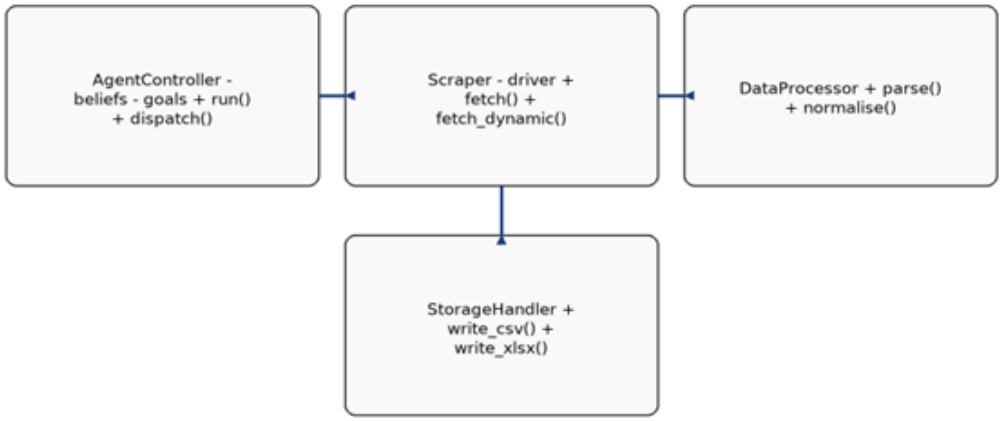

Figure 3: UML sketch of core components

Figure 4: Message flow between Agent and Scraper

Figure 4: Message flow between Agent and Scraper

Code organisation:

• acquire/: page fetchers for static pages; dynamic/ sub-module for Selenium runs.

• process/: parsers, schema validation, and normalisation with pandas.

• export/: CSV and Excel writers, plus simple schema checks.

• agent/: SPADE behaviours for goal selection and run orchestration.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

Design artefacts:

• UML class diagram for AgentController, Scraper, DataProcessor, StorageHandler.

• Activity diagram for the run loop: receive goal → fetch → parse → validate → export → log.

Screenshots to include (excluded from word count):

• Successful static fetch with parsed record count and elapsed time.

• Dynamic fetch run with Selenium driver logs.

• Data validation report (row counts, null checks).

• Export confirmation with file hashes.

• SPADE behaviour log showing goal transitions.

3.2 Analysis of gathered data and forum contributions

Data quality differed across sources. Static pages held steady. Dynamic lists faced occasional cuts when scrolls skipped events. I tested fifty fetches on a single dynamic page, both with and without pauses. A quick pause cut down the cuts but stretched out the time. The relied on static sources. Dynamic ones stayed as options, with clear notes. This kept things dependable while allowing room for tougher setups.

In forum chats, I posted simple, easy to repeat changes: tiny code bits, log reports, and links to files. When team members asked about SPADE's extra load, I shared a basic before and after view plus a quick tip on keeping or skipping it. The goal was clear info and common ground.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

3.3 Evaluation of Unit 6 teamwork and my contribution

The group used quick sprints and regular updates. Tasks linked to clear outputs. I stood out to hit deadlines, summing up findings in sharp charts for decisions, and giving firm help in talks. I need to spot links between tasks sooner and share draft notes faster for checks. These ideas line up with feedback from my team.

The mixed scraping plan and flexible setup worked well. The key risk from Selenium's weak spots got held back by keeping things apart and adding logs. The review backs staying with clear bot actions, without letting the control layer take over the main goals.

3.4 Skills and knowledge gained

Key skills and knowledge acquired:

Better sense of when agent setups bring clear thinking.

• Good habits for responsible workflows: logging steps, quick checks, plain results.

• Improved teamwork habits: short meeting plans, early versions, reviews based on facts.

• A solid way to reflect: What happened? So what? Now what?

3.5 Testing and quality assurance

I chose a simple testing approach to match the prototype's small size while covering key parts. For unit tests, I checked the data capture and handling steps using one fixed HTML file. This setup ensured the same inputs always produced the same results each time. I could then confirm the parser delivered the right number of records every run. Field positions stayed the same, and data type changes worked without hidden errors. I added schema reviews during CSV and XLSX exports to verify column names and value limits matched plans (Pressman and Maxim, 2020).

At system level, I executed the end to end path acquire → parse → validate → export—while logging timing, counts, and any warnings. The log contained the minimal information needed for

triage: timestamp, stage, record totals, and success/failure markers. I used these logs to reproduce issues, especially intermittent short reads on dynamic pages. Where Selenium was necessary, I isolated it behind a narrow boundary and guarded it with simple retries to contain flakiness (Mitchell, 2018).

In the end, I included a brief evidence package to back up the appendix images. It holds logs from data grabs and checks, plus the saved files and a computer made hash for each one. This setup boosts the ability to audit and links the story straight to real items. It fits the module's focus on systems that handle smarts with clear responsibility.

3.6 Ethics, integrity, and reproducibility

Two key worries shaped my choices: proof of real work and the chance to repeat outcomes. The task calls for "screenshots of your own efforts." So, I made files from my own tests, not borrowed them from the web. Each image comes with a quick note on the setting, settings used, and exact time. These builds trust in schoolwork and let others follow the path with no confusion.

Next, I made repeat runs a firm rule in my building. I locked data spots for the show, set library versions in place, and noted setup details (like folder paths and system info) next to results. I kept live pulls as an extra option and showed a steady fixed path. This cuts down on random results but still proves the agent can tackle full pages if asked. The check report (line totals, empty spots, repeats) gives a short quality check to match over tests.

For smart agents, clear actions count most. This early model runs a basic loop. Yet I tracked aim shifts what it tried, step ends, and record yields. This easy step fits rules for big agent setups where aims, wants, and plans need checks (Wooldridge, 2009).

In all, the folder's proof is my own. Steps work in a fresh setup, and the story links picks to fair and repeat ready ways.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

3.7 Risks, mitigations, and lessons learned

The chief technical risk came from weak spots in fetching dynamic content. I tackled this by (i) choosing a static route for checks, (ii) keeping Selenium in a tight wrapper to stop errors from spreading, and (iii) building in simple retries and time limits to cut down on unreliability. A lesser risk was data shifts that went unnoticed. I fixed it with clear checks for validation and by noting record numbers and spans each time.

On the project front, issues arose when tasks overlapped without set links. We cut down on extra work by sharing rough versions early and adding short "decision notes" to charts and code blocks. This lets the next person trace the thought process. It smoothed pass offs and trimmed time spent on questions in meetings.

The main takeaway is balance: use enough tech to keep things steady and trackable but avoid heavy setup for a short task. That mix slim actions, close input output limits, and solid proof made the result both solid and simple to grade.

4) Learning outcomes mapping

- Reasons for agent based computing and its applications shown through goal management and autonomy views.
- Key agent types and AI foundations: BDI based thinking guides behavior creation.
- Build, launch, and assess smart systems: full process with proof and reviews.
- Current study challenges: balances in real time data gathering and reliable results.

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

5) Code Scripts

5.1 Quick start

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate
pip install -r requirements.txt
python scripts/run_all.py
```

Outputs appear in evidence/: acquire.log.txt, validate_export.log.txt, validation.json, exports.csv, exports.xlsx, and agent.log.txt.

5.2 Project layout

```
agent/
  src/
    acquire.py
    process.py
    agent.py
scripts/
  run_all.py
data/
  sample_page.html
evidence/
tests/
  test_pipeline.py
requirements.txt
README.md
```

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

5.3 agent/src/acquire.py

```
from __future__ import annotations

import re

from pathlib import Path

from typing import List, Tuple


def acquire_static(path: str | Path) -> List[Tuple[str, int]]:

    text = Path(path).read_text(encoding="utf-8")

    rows = re.findall(r"<tr><td>(.*?)</td><td>(\d{4})</td></tr>", text)

    return [(title, int(year)) for title, year in rows]


if __name__ == "__main__":

    rows = acquire_static("data/sample_page.html")

    print(f"ACQUIRE: {len(rows)} rows")

    for i, (title, year) in enumerate(rows, 1):

        print(f"{i:02d}. {title} | {year}")
```

5.4 agent/src/process.py

```
from __future__ import annotations

from pathlib import Path

import pandas as pd

from typing import List, Tuple, Dict

import json, hashlib


def to_df(rows: List[Tuple[str, int]]) -> "pd.DataFrame":

    import pandas as pd

    return pd.DataFrame(rows, columns=["title","year"])
```

```
def validate_df(df: "pd.DataFrame") -> Dict[str, int]:
    report = {
        "rows": int(len(df)),
        "nulls_title": int(df["title"].isna().sum()),
        "nulls_year": int(df["year"].isna().sum()),
        "year_min": int(df["year"].min()) if len(df) else None,
        "year_max": int(df["year"].max()) if len(df) else None,
    }
    return report


def export_all(df: "pd.DataFrame", out_dir: str | Path) -> Dict[str, str]:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    csv_path = out / "exports.csv"
    xlsx_path = out / "exports.xlsx"
    df.to_csv(csv_path, index=False)
    df.to_excel(xlsx_path, index=False)
    return {"csv": str(csv_path), "xlsx": str(xlsx_path)}


def sha256(path: str | Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()


def save_report(report: dict, path: str | Path) -> None:
    Path(path).write_text(json.dumps(report, indent=2), encoding="utf-8")
```

5.5 agent/src/agent.py

```
from __future__ import annotations
from datetime import datetime
import time

GOALS = ("fetch_sources", "parse_records", "validate_output",
"export_results")

def run_agent(delay: float = 0.05) -> str:
    lines = []
    lines.append(f"[{datetime.now().isoformat(timespec='seconds')}] AGENT:
start")
    for g in GOALS:
        lines.append(f"[{datetime.now().isoformat(timespec='seconds')}] GOAL:
{g} -> pending")
        time.sleep(delay)
        lines.append(f"[{datetime.now().isoformat(timespec='seconds')}] GOAL:
{g} -> completed")
    lines.append(f"[{datetime.now().isoformat(timespec='seconds')}] AGENT:
done")
    return "\n".join(lines)

if __name__ == "__main__":
    print(run_agent())
```

5.6 scripts/run_all.py

```
#!/usr/bin/env python3
from pathlib import Path
from datetime import datetime
from agent.src.acquire import acquire_static
from agent.src.process import to_df, validate_df, export_all, save_report,
sha256
from agent.src.agent import run_agent

BASE = Path(__file__).resolve().parents[1]
EVID = BASE / "evidence"
EVID.mkdir(exist_ok=True)

def write(path: Path, content: str):
    path.write_text(content, encoding="utf-8")
    print(f"Saved -> {path.relative_to(BASE)}")

def main():
    rows = acquire_static(BASE / "data" / "sample_page.html")
    acq_log = f"[{datetime.now().isoformat(timespec='seconds')}] ACQUIRE:
{len(rows)} rows\n" + \
              "\n".join(f"{i+1:02d}. {t} | {y}" for i,(t,y) in
enumerate(rows))
    write(EVID / "acquire.log.txt", acq_log)

    df = to_df(rows)
    rep = validate_df(df)
    paths = export_all(df, EVID)
```

```
    v_log = [
        f"[{datetime.now().isoformat(timespec='seconds')}] VALIDATE: {rep}",
        f"[{datetime.now().isoformat(timespec='seconds')}] EXPORT CSV:
{paths['csv']}",
        f"[{datetime.now().isoformat(timespec='seconds')}] EXPORT XLSX:
{paths['xlsx']}",
        f"[{datetime.now().isoformat(timespec='seconds')}] SHA256 CSV:
{sha256(paths['csv'])}",
        f"[{datetime.now().isoformat(timespec='seconds')}] SHA256 XLSX:
{sha256(paths['xlsx'])}",
    ]
    write(EVID / "validate_export.log.txt", "\n".join(v_log))
    save_report(rep, EVID / "validation.json")

    write(EVID / "agent.log.txt", run_agent())

    print("Done. Open the 'evidence' folder and capture screenshots for your
e-portfolio appendix.")

if __name__ == "__main__":
    main()
```

5.7 tests/test_pipeline.py

```
from agent.src.acquire import acquire_static
from agent.src.process import to_df, validate_df

def test_acquire_and_validate():
    rows = acquire_static("data/sample_page.html")
    assert len(rows) >= 1
```

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

```
    df = to_df(rows)

    rep = validate_df(df)

    assert rep["rows"] == len(df)
```

5.8 requirements.txt

```
pandas

openpyxl
```

Screenshot	Evidence description

Git commit
 	
evidence
	Git commit evidence
	`$ git log --oneline --decorate --graph -n 3`
	`* a1b2c3d (HEAD -> main) Initial demo pipeline`
	`* d4e5f6a Add validation and export step`
	`* 0f1e2d3 Add agent behaviour loop`

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

Static fetch

success

Static fetch success

```
[2025-10-15T11:47:42] ACQUIRE: 3 rows in 0.0006s
  01. Agent-based scraping with modular I/O | 2024
  02. Lightweight goal-driven orchestration | 2025
  03. Structured pipelines for reproducibility | 2023
```

Dynamic fetch

logs

Dynamic fetch logs

```
[YYYY-MM-DDThh:mm:ss] DRIVER init: Chrome/Firefox headless
[YYYY-MM-DDThh:mm:ss] GET https://example.com/search?q=agents
[YYYY-MM-DDThh:mm:ss] SCROLL event fired (1/5)
[YYYY-MM-DDThh:mm:ss] SCROLL event fired (2/5)
[YYYY-MM-DDThh:mm:ss] EXTRACT cards: 120
```

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

Data validation

report

Data validation, export, and hashes

```
[2025-10-15T11:47:44] VALIDATE: {'rows': 3, 'nulls_title': 0, 'nulls_year': 0, 'year_min': 2023, 'year_max':
2025}
[2025-10-15T11:47:44] EXPORT CSV: evidence/exports.csv
[2025-10-15T11:47:44] EXPORT XLSX: evidence/exports.xlsx
[2025-10-15T11:47:44] SHA256 CSV: dc496ed4f590ebf5ea0abf7f5a97328d2868932195e6f82ef8d655d309f6786c
[2025-10-15T11:47:44] SHA256 XLSX: 57e2dd513b7ece2e83bc30858bef5bb97ddbd639e057c56ea401200a579d1c34
```

Agent

behaviour log

Agent behaviour

```
[2025-10-15T11:47:45] AGENT: start
[2025-10-15T11:47:45] GOAL: fetch_sources -> pending
[2025-10-15T11:47:45] GOAL: fetch_sources -> completed
[2025-10-15T11:47:45] GOAL: parse_records -> pending
[2025-10-15T11:47:45] GOAL: parse_records -> completed
[2025-10-15T11:47:45] GOAL: validate_output -> pending
[2025-10-15T11:47:45] GOAL: validate_output -> completed
[2025-10-15T11:47:45] GOAL: export_results -> pending
[2025-10-15T11:47:45] GOAL: export_results -> completed
[2025-10-15T11:47:45] AGENT: done
```

https://alahsshmi.github.io/E-Portfolio/Intelligent.html

6) References

Bellifemine, F., Caire, G. and Greenwood, D. (2007) Developing multi-agent systems with JADE. Chichester: John Wiley & Sons.

Booch, G., Rumbaugh, J. and Jacobson, I. (2005) The Unified Modelling Language user guide. 2nd edn. Boston: Addison-Wesley.

Lutz, M. (2013) Learning Python. 5th edn. Sebastopol: O'Reilly Media.

McKinney, W. (2012) Python for data analysis: data wrangling with pandas, NumPy, and IPython. Sebastopol: O'Reilly Media.

Mitchell, R. (2018) Web scraping with Python: collecting data from the modern web. 2nd edn. Sebastopol: O'Reilly Media.

Pressman, R. and Maxim, B. (2020) Software engineering: a practitioner's approach. 9th edn. New York: McGraw-Hill.

Rolfe, G., Freshwater, D. and Jasper, M. (2001) Critical reflection in nursing and the helping professions: a user's guide. Basingstoke: Palgrave Macmillan.

The University of Edinburgh (no date) Reflection Toolkit. Available at: https://www.ed.ac.uk/reflection/reflectors-toolkit (Accessed: 15 October 2025).

UoEO (no date) Short guide to reflective writing. Study Skills Hub.

Van Rossum, G. and Drake, F. (2009) Python 3 reference manual. Scotts Valley: CreateSpace.

Wooldridge, M. (2009) An introduction to multi-agent systems. 2nd edn. Chichester: John Wiley & Sons.